Git Workflow Guide

PSID Project

Sarah Sullivan

Created: March 25, 2026

Last Updated: March 25, 2026

Made with: GitHub Copilot / Claude Sonnet 4.6

This file summarizes the command line statements to push a commit to GitHub.

VS Code setting: `settings.json` is configured with `latex-workshop.latex.autoClean.run: "onBuilt"` — auxiliary `.tex` files (`.aux`, `.log`, `.toc`, etc.) are automatically deleted after every compilation.

Contents

I. Workflow

1) Everyday: push VS Code to GitHub

```
git pull                          # get latest changes
git add .                         # stage changes
git commit -m "comment changes"
git push                          # upload to GitHub
```

2) Collaborating: push GitHub changes to VS Code

```
git pull origin main
git pull
```

II. Set Up

1) Login

```
git config --global user.name "Your Name"
git config --global user.email "sbsullivan19@gmail.com"
```

2) Navigate to repo folder

```
cd "/Users/sarsul/Library/CloudStorage/Dropbox-
   UniversityofMichigan/Sarah Sullivan/SARAH-SOFT/research/psid"
```

3) Initialize

```
git init
git branch -M main
```

4) Edit `.gitignore`

Open `.gitignore` in VS Code:

```
code .gitignore
```

Add files and folders to exclude from the repository. Currently, I am only uploading `.do` files to the repository.

After editing, stage and commit `.gitignore`:

```
git add .gitignore
git commit -m "update gitignore"
```

If files were **already tracked** by Git before you added them to `.gitignore`, Git will keep tracking them. Untrack a specific file without deleting it:

```
git rm --cached filename.dta
```

Untrack everything and re-add (use carefully):

```
git rm --cached -r .
git add .
git commit -m "remove tracked files now in gitignore"
```

5) Stage and commit

```
git add .
git status              # check what will be committed
git commit -m "COMMIT MESSAGE"
```

6) Connect to remote (first time only)

```
git remote add origin https://github.com/sarahs-house/psid.git
```

Check it worked:

```
git remote -v
```

7) Push to GitHub

```
git push -u origin main
```

After the first push, you can just use:

```
git push
```

III. Troubleshooting

1) If push is rejected (histories don't match)

```
git fetch origin
git rebase origin/main
```

Or if you need to force-merge unrelated histories:

```
git pull origin main --allow-unrelated-histories
git push -u origin main
```

2) Lock file error

```
fatal: Unable to create '.git/index.lock': File exists.
```

Fix:

```
rm -f .git/index.lock
```

3) Unstage a file

```
git restore --staged filename.do
```

4) Check status / history / remotes

```
git status
git log --oneline
git remote -v
```

5) See what changed

```
git diff
```

IV. VS Code Command Line

1) Open files & folders

```
code .                              # open current folder in VS Code
code filename.do                    # open a specific file in VS Code
open filename.pdf                   # open file in default app (e.g.
    PDF viewer)
open -a "TeXShop" file.tex          # open file in a specific app
```

2) Navigate directories

```
pwd                                 # print current directory
ls                                  # list files in current directory
ls -la                             # list files with details and
    hidden files
cd foldername                      # go into a folder
cd ..                               # go up one level
cd -                                # go back to previous directory
```

3) Compile LaTeX

```
pdflatex filename.tex            # compile .tex to PDF (run twice
    for TOC)
```

4) File operations

```
mv oldname.do newname.do         # rename a file
cp file.do copy_of_file.do       # copy a file
rm filename                      # delete a file (careful -- no
    undo!)
mkdir foldername                 # create a new folder
```